

Integer factorization

In this article we list several algorithms for the factorization of integers, each of which can be either fast or varying levels of slow depending on their input.

Notice, if the number that you want to factorize is actually a prime number, most of the algorithms will run very slowly. This is especially true for Fermat's, Pollard's $p-1$ and Pollard's rho factorization algorithms. Therefore, it makes the most sense to perform a probabilistic (or a fast deterministic) [primality test](#) before trying to factorize the number.

Trial division

This is the most basic algorithm to find a prime factorization.

We divide by each possible divisor d . It can be observed that it is impossible for all prime factors of a composite number n to be bigger than $\sqrt{n}$. Therefore, we only need to test the divisors $2 \leq d \leq \sqrt{n}$, which gives us the prime factorization in $O(\sqrt{n})$. (This is [pseudo-polynomial time](#), i.e. polynomial in the value of the input but exponential in the number of bits of the input.)

The smallest divisor must be a prime number. We remove the factored number, and continue the process. If we cannot find any divisor in the range $[2; \sqrt{n}]$, then the number itself has to be prime.

```
vector<long long> trial_division1(long long n) {
    vector<long long> factorization;
    for (long long d = 2; d * d <= n; d++) {
        while (n % d == 0) {
            factorization.push_back(d);
            n /= d;
        }
    }
    if (n > 1)
        factorization.push_back(n);
    return factorization;
}
```